Change Management Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC8.1 (Change management) · CC7.1 (Design + implementation of controls) · ISO 27001 A.8.32 · HIPAA §164.312(a)(1) reasonable safeguards **Companion documents:** Access Control Policy · Incident Response Policy · BC/DR Policy · Encryption Policy §7.

1. Purpose

Describe how changes actually get from a developer's box to production without breaking customer trust. This is not aspirational — it's the pipeline we actually run today, written down so a reviewer can trace any deployed byte back to a specific commit + specific test result + specific human decision.

2. Scope

- **Code:** every commit to `railcall-core`, `railcall-engine`, `railcall-contrib`.
- **Station releases:** every station tarball published on GitHub Releases + mirrored to railcall.ai.
- **Gateway deploys:** every Render auto-deploy triggered by push to `railcall-core:main`.
- **Website deploys:** every pm2 restart on VPS `157.230.177.45`.
- **Configuration changes:** env-var edits on Render, DNS record edits on Cloudflare, Stripe Price/Product edits, GitHub org / repo settings.

OUT of scope: local dev-environment tweaks that never leave a developer's box.

3. Change classes

Class	Definition	Example	Approval	Testing gate
Standard	Additive, no state migration, no protocol change	Adding a new CLI command, docs edits, new test	Author self-review	Existing test suite must pass
Enhancement	Behavior change, backwards-compatible	Adding seat_count column with NULL default; new endpoint that returns 401 for anonymous	Author + one reviewer (Sami if Claude authored)	Existing + new dedicated tests
Breaking	Not backwards-compatible; requires customer action	Rotating ISSUER_PUBKEY_HEX; changing entitlement schema; removing an endpoint	Sami explicit sign-off + release ceremony	Full regression + customer-facing communication plan
Emergency	Fix in production for an active incident	Rolling back a bad deploy; hotfix for exploited vuln	IC decision, retroactive review within 24 h	Minimum viable test at deploy time; full test in follow-up PR

4. The pipeline — what actually runs

For a Standard or Enhancement class change to `railcall-core`:

1. **Code + tests locally.** Author writes the change and at least one dedicated test that would have failed pre-change and passes post-change.
2. **Local test suite.** `python3 -m pytest tests/ -q` + any suite specific to the file touched (e.g., `test_seat_checkin.py` for seat code, `test_paid_full_e2e.py` for full-chain changes).
3. **Commit with an evidence-forward message.** Message describes WHY (the invariant being defended or the gap being closed), not just WHAT.
4. **Push to `railcall-core:main`.** Triggers Render auto-deploy.
5. **Watch the deploy.** Render dashboard shows deploy status. Deploy failures roll back automatically.
6. **Post-deploy smoke.** `curl` the affected endpoint(s) with a known-shape request; verify expected HTTP code + response shape. Log the smoke result to the incident note if the change

is Enhancement or above.

For `railcall-engine`:

1. Same 1–3 above.
2. Push to the working branch (`feat/paid-tier-entitlement` currently). Engine does NOT auto-deploy; changes reach users only via a station release (§5).

For `railcall-contrib` (website + installer mirror):

1. Same 1–3.
2. Push to `feat/website-v2`.
3. `ssh sami@157.230.177.45 && cd ~/railcall-contrib && git pull`.
4. If a `.tsx` / `.ts` changed: `cd website-v2 && npm run build`.
5. If a file under `website-v2/public/` changed: `pm2 restart railcall-website` (Next.js caches the public/ listing at boot; changed files need a restart to serve).
6. Verify the affected URL via `curl` from outside the VPS.

---

5. Station release ceremony (Breaking-class only)

Station releases are the highest-blast-radius change class we make. Every user's install eventually runs the code we cut here.

Full ceremony (mirrors what was executed for `station-v0.16` on 2026-07-21):

1. **Verify engine is on the right branch + commit** (`feat/paid-tier-entitlement` currently, but check per-release).
2. **Overlay engine into station tree:** `rsync -a --exclude='__pycache__' --exclude='*.pyc' --exclude='tests/' --exclude='.railcall_workspace/' "$ENGINE/workbench/" "$STATION/workbench/"`.
3. **Verify pin (if rotating):** `grep '^ISSUER_PUBKEY_HEX' "$STATION/workbench/primitives/entitlement.py"` matches the intended value.
4. **Build tarball:** `STATION_SRC=... OUT=... RELEASE_TAG=station-vN ENGINE_COMMIT=... CORE_COMMIT=... bash scripts/build_station_tar.sh`.
5. **Verify leak gate passed** — the build script refuses to publish if any factory/moat artifact is included. Non-optional.
6. **Verify tarball has no secrets:** `tar -tzf <tarball>` and grep for `keys.local.json`, `.env`, `token.json` — must return zero matches.
7. **Boot the extracted station on an unused port** (`STUDIO_PORT=8815 python3 studio_server.py`), hit `/api/version` — must return the new `release_tag`.

8. **sha256 the tarball.** `shasum -a 256 "$OUT"`.
9. **Publish to GitHub Releases:** `gh release create station-vN "$OUT" --title "..." --notes "..."`.
10. **Rename the asset to `railcall_station.tar.gz`** (unversioned, per install.sh URL convention).
11. **Byte-compare downloaded release asset against local build.** Must match. If not: DO NOT PIN.
12. **Update `STATION_SHA` + `STATION_URL` in `install.sh` AND `install.ps1`.**
13. **Commit + push `railcall-core`.**
14. **Mirror to contrib:** `cp` installer + tarball into `railcall-contrib/website-v2/public/`, commit, push `feat/website-v2`.
15. **Deploy website:** `ssh sami@... && cd ~/railcall-contrib && git pull && pm2 restart railcall-website`.
16. **Verify public endpoints:** `curl https://railcall.ai/install.sh | grep STATION_SHA` matches the new sha; `shasum -a 256 <(curl https://railcall.ai/railcall_station.tar.gz)` matches.
17. **Fresh-install smoke:** in an isolated `HOME`, `curl -fsSL https://railcall.ai/install.sh | HOME=... bash`. Run `railcall version` — must show the new `station-vN ✓ latest`.

This ceremony is documented step-for-step because it's the one an operator under stress has to follow exactly. Any deviation → don't ship.

---

6. Approval + separation of duties

Small team reality: Sami is often the author AND the reviewer AND the deployer. That's not a separation-of-duties failure if it's honestly documented. What IS required:

- **Machine-enforced gates precede human-enforced ones.** Tests + leak gate + sha byte-compare are the actual safety net; the human review confirms them, doesn't substitute.
- **Standing merge authorization** (per `feedback_merge_authorization.md`): merges on green CI across repos Sami controls proceed without asking each time. This is a documented, deliberate policy for a two-person team.
- **Breaking changes still require Sami's explicit sign-off** even under standing auth. See §3.
- **Emergency changes require post-hoc review** within 24 h. Log in the incident note; document in the next weekly sync.

When headcount grows past 2, this section should tighten — separate authors from approvers on Breaking-class + revisit standing auth.

7. Rollback

Every deploy has a rollback path. Documented per surface:

- **Render gateway:** Render dashboard → "Manual Deploy" → pick the previous known-good deploy. ~2 min.
- **Station release:** revert the `install.sh` pin change (commit + push); users' next install picks up the previous tarball. Users who already installed the bad version need a reinstall (`curl -fsSL https://railcall.ai/install.sh | bash`).
- **VPS website:** `ssh sami@... && cd ~/railcall-contrib && git reset --hard <prev-commit> && npm run build && pm2 restart railcall-website`.
- **Engine commits:** git revert on the working branch. Only reaches production via a subsequent station release, so effectively delayed until §5 is repeated.
- **Stripe / Render env-var change:** re-edit the variable to the previous value; save; wait for redeploy.

Rollback is always preferable to "roll forward through the fire" during an active incident. Take rollback first, root-cause second.

8. Emergency changes — exceptions to §4

An Emergency change bypasses parts of §4 to restore service. Constraints:

- **Only the IC can invoke emergency mode.** Log the invocation in the incident note.
- **Skip is documented** — every skipped step (tests, review, ceremony sequence) is listed in the incident note with the reason.
- **Follow-up PR within 24 h** brings the code back into full compliance (tests written, formal review, any missed docs updated).
- **Post-mortem** covers the emergency change specifically: was the shortcut justified, could it have been avoided.

Examples of legitimate emergency: rolling back an install.sh pin that broke every fresh install. Not emergency: skipping tests because the fix is "obvious."

9. Post-change verification

For every Enhancement + Breaking change, post-deploy verification includes:

1. **Endpoint smoke:** hit affected URLs, verify expected responses.

2. **Log check:** watch Render logs (or `pm2 logs`) for the first 5 min post-deploy. Any new error class → investigate.
3. **Update any relevant Probo controls or measures** — a change that closes a gap should flip that control's status from `NONE` to `INITIAL` / `MANAGED` / `DEFINED` (see Risk Management Policy for the state machine).

Every deployment leaves an audit trail: the git commit is the change record; the Render deploy is the deploy record; the smoke verification is captured in the commit message (for Standard) or an incident note (for Emergency).

---

10. Configuration change tracking

Not everything is in Git. What's not:

Change	Where it lives	How we track it
Render env vars (all `PROBOD_*`, `RAILCALL_*`, `STRIPE_*`, `DOMAIN_URL`)	Render dashboard	Documented in `render.yaml` (variable names + `sync:false` marker); values held in 1Password + Render itself
DNS records	Cloudflare	Cloudflare's own change log
Stripe Products + Prices	Stripe dashboard	Stripe's own audit log; Price IDs referenced in code
GitHub org settings	GitHub	GitHub audit log

Rule: if a value in one of these systems changes and code doesn't, the change is still a Change under this policy — document in the incident note or a "config-change" issue in the repo.

---

11. Review

Whole policy reviewed at each station release + at least annually. §5 (station release ceremony) is the section most likely to drift — if we automate any step, this document is the source that must be updated first, not last.

---

12. Related documents

- Incident Response Policy §8 — the emergency-change escalation path.
- BC/DR Policy §5.1 + §5.4 — the recovery paths that station release + VPS deploy runbooks share with change management.

- Access Control Policy §6 — who has the credentials each deploy step requires.
- Vendor Management Policy §3.1, §3.5 — Render + VPS specifics.